

OOPs

The key attributes of Object-Oriented Programming (OOP)

- The Four Pillars i.e (PAIE)
 - **Polymorphism**
 - **Abstraction,**
 - **Inheritance**
 - **Encapsulation**
- These principles, along with fundamental concepts like Classes and Objects, enable to create modular, reusable, and scalable code.

Encapsulation

- Protective shield- data integrity and security
- bundling the data (attributes) and the methods that operate on that data into a single unit (a class)

To achieve Encapsulation

- **Declare data members as private:** Restricts direct access to the data from outside the class.
- **Provide public methods** (getters and setters): These methods act as intermediaries, allowing controlled access to the data.
 - Getters retrieve data values,
 - while setters modify them.
- **Implement data validation:** Within the setter methods, you can validate data before assigning it to the object's properties, ensuring data integrity

Components

- **Data Hiding**

- Restricting access to data

- Data hiding protects an object's internal data from **unauthorized or accidental access** by making data members **private** and exposing them only through **public methods**.

- (Hide the data, expose the behavior")

Access modifiers

Modifier	Access Level
private	Accessible only within the same class
protected	Same package or subclass
public	Accessible everywhere

These keywords determine the accessibility of classes, methods, and other members.

Default modifier

- Omitting an access modifier for a class, method, or data member grants access only within the current package.

```
class MyClass {  
    int myVariable; // Default access modifier  
    void myMethod() {  
        System.out.println("This is a default method");  
    }  
}
```

```
class BankAccount {  
    private double balance; // Data hidden  
  
    public void setBalance(double amount) {  
        if (amount > 0) {  
            balance = amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

- balance cannot be accessed directly
- Access is controlled via setBalance() and getBalance()

Real-Life Example

- **ATM Machine**
 - You cannot access bank data directly
 - You interact using buttons (methods)

Encapsulation Vs Data Hiding

Encapsulation

Wrapping data & methods

Conceptual mechanism

Achieved using classes

Data Hiding

Restricting data access

Security mechanism

Achieved using access modifiers

Accessibility Matrix

- Getters and Setters
 - Getters and Setters are methods for accessing and updating private variables in a class.
 - Programmers often declare variables as “private” to protect data.
 - **Getters and Setters protect data, making your code more secure**

Getter Method

- Retrieves a private variable's value from within a class.
- It provides controlled access to variables.
- The method name typically begins with “get” followed by a capitalized variable name, such as “getName”.

```
public class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}
```

- The given code defines a Person class with a private name variable and a getName() method.
- The getName() method returns the name variable.

Setter Method

- Updates or sets the value of a private variable.
- The method begins with the word “set” followed by the variable name, with the first letter capitalized (e.g., setName).

```
public class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
  
    public void setName(String newName) {  
        name = newName;  
    }  
}
```

- The above code has a method called setName, which lets you change the value of the name variable.

Polymorphism

- one interface, many forms
- allows the same method or operation to behave **differently for different objects**, even though they share a common interface.
 - Polymorphism enables objects of different classes to respond to the **same method call in their own way**.

Real-Life Example

- Consider a **payment system**:
 - Credit Card → processes payment differently
 - Debit Card → processes payment differently
 - UPI → processes payment differently
- The action is the same (**makePayment()**), but the behavior varies.

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Animal a;  
        a = new Dog();  
        a.sound(); // Dog barks  
        a = new Cat();  
        a.sound(); // Cat meows  
    }  
}
```

Types of Polymorphism

- **Compile-time (Static)**
 - Method overloading
 - Same method name, different parameters
- **Run-time (Dynamic)**
 - Method overriding
 - Achieved using inheritance and dynamic binding

Compile-time Polymorphism(Static)

- **multiple behaviors are defined at compile time** using the same method name but with **different parameters or forms**.
- The compiler decides **which method to call** based on:
 - Number of parameters
 - Type of parameters
 - Order of parameters

Compile time Poly

- **Method Overloading**
- **Operator Overloading (in some languages like C++)**

Method Overloading

- Same method name, **different parameter lists**, within the same class.

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

The compiler decides which add() method to call based on arguments.

Operator Overloading

- Allows operators to behave differently for user-defined data types.

```
class Complex {  
public:  
    int real, imag;
```

```
    Complex(int r, int i) {  
        real = r;  
        imag = i;  
    }
```

```
    Complex operator + (Complex const  
&obj) {  
        return Complex(real + obj.real, imag +  
obj.imag);  
    }  
};
```

Advantages

- Faster execution (no runtime overhead)
- Better readability
- Code reusability
- Note-Compile-time polymorphism provides **multiple method definitions**
 - Decisions made **during compilation**
 - Implemented using **method overloading** and **operator overloading**

Run-time Polymorphism(Dynamic)

- It allows a **derived (child) class** to provide a **specific implementation** of a method that is already defined in its **base (parent) class**.
- The method that gets executed is determined by the **object**, not the reference type.
 - **Method Overriding**
 - **Inheritance**
 - **Dynamic Binding**

Method Overriding

- A child class provides its own version of a method defined in the parent class with:
 - Same method name
 - Same parameters
 - Same return type (or compatible)

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog(); // Upcasting  
        a.sound();           // Calls Dog's method  
    }  
}
```

Dog barks

Static vs Dynamic Polymorphism

Compile-time

Run-time

Method overloading

Method overriding

Faster

Slower

No inheritance
required

Requires inheritance

Binding at compile
time

Binding at runtime

Abstraction

- Reduces complexity by focusing on **what** a system does rather than **how** it does it.
- It presents only the **essential features** to the user while hiding unnecessary internal details.

- when driving a car, the driver interacts only with the **steering wheel, accelerator, brake, and clutch**—these form the **abstract interface**.
- The driver does not need to understand the complex internal mechanisms of the **engine, gearbox, or transmission system**, which remain hidden.
- This simplification allows the user to operate the car efficiently without being concerned about its internal working.

- It is achieved using **abstract classes and interfaces.**

Abstract Class (Java)

- An abstract class can have:
 - Abstract methods (without body)
 - Concrete methods (with body)


```
abstract class Vehicle {  
    abstract void start(); // abstract method  
  
    void fuel() {  
        System.out.println("Vehicle needs fuel");  
    }  
}
```

```
class Car extends Vehicle {  
    void start() {  
        System.out.println("Car starts with key");  
    }  
}
```

Interface (Java)

- An interface provides **100% abstraction**

```
interface Payment {  
    void pay(); // abstract method  
}
```

```
class UPI implements Payment {  
    public void pay() {  
        System.out.println("Payment through UPI");  
    }  
}
```

User



[Interface / Abstract
Class]



Implementation Details
(hidden from user)

Advantages of Abstraction

- Reduces complexity
- Improves security
- Enhances flexibility
- Increases code reusability

Inheritance

- A class acquires the properties and behaviors of another class.
- Inheritance allows a child class (subclass) to inherit data members and methods of a parent class (superclass).
- It helps in code reusability, hierarchical classification, and extensibility.

Example

- Relationship: **“IS-A”**
 - Dog **is an** Animal
 - Car **is a** Vehicle

```
class Parent {  
    void show() {  
        System.out.println("Parent class");  
    }  
}
```

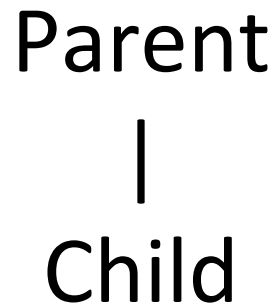
```
class Child extends Parent {  
    void display() {  
        System.out.println("Child class");  
    }  
}
```

Types of Inheritance

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance
- Hybrid Inheritance

Single Inheritance

- One child class inherits properties and methods from only one parent class.



Example

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
class Dog extends Animal {  
    void eat() {  
        System.out.println("Dog eats");  
    }  
}
```

Multilevel Inheritance

- allows a subclass to inherit properties and methods from a superclass **through another subclass**, creating multiple levels.

Grandparent Class

|

Parent Class

|

Child Class

Example

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Puppy extends Dog {  
    void play() {  
        System.out.println("Puppy plays");  
    }  
}
```